Active Circuit Discovery: A Multi-Action POMDP Agent for Causal Feature Identification in Transformer Attribution Graphs

Sharath Sathish [1], Mominul Ahsan [2] and Majid Latifi [2,*]

[1] *BP International Limited, London TW16 7BP, UK; sharath.sathish@gmail.com*
[2] *Department of Computer Science, University of York, York YO10 5GH, UK; mominul.ahsan2@gmail.com; majid.latifi@york.ac.uk*

Correspondence: majid.latifi@york.ac.uk

Abstract: Mechanistic interpretability seeks to reverse-engineer the computational circuits within large language models, yet current methods rely on exhaustive or heuristic search over exponentially many feature interactions. This paper introduces *Active Circuit Discovery* (ACD), a framework that combines attribution graph analysis with active inference for efficient circuit discovery. The framework integrates Anthropic's `circuit-tracer` library as the attribution graph backend, using Edge Attribution Patching with transcoders to identify active transcoder features per prompt. A POMDP agent, implemented with `pymdp`, maintains a multi-factor generative model of feature importance, layer role, and causal influence, and selects both the target feature and intervention type (ablation, activation patching, or feature steering) at each step by minimising Expected Free Energy over the joint feature–action space. The agent learns its observation model online through Dirichlet parameter updates, enabling principled exploration–exploitation trade-offs in circuit discovery. The framework is evaluated on two architectures, Gemma-2-2B (26 layers) and Llama-3.2-1B (16 layers), across four settings: Indirect Object Identification (IOI), multi-step reasoning, feature steering, and a multi-domain benchmark spanning geography, mathematics, science, logic, and history. With a budget of 20 interventions per prompt, the POMDP agent achieves 1255% oracle efficiency on Gemma IOI and 90.6% on Llama IOI, substantially outperforming random, greedy, and bandit baselines on both architectures. Feature steering confirms causal controllability: scaling individual features at $10\times$ activation changes predictions for 24 of 200 tested features across both models ($p < 10^{-15}$, binomial test). Multi-domain analysis reveals task-dependent circuit structure: IOI circuits concentrate in late layers, while reasoning and scientific knowledge recruit early and middle layers. Code, notebooks, and experiment results are publicly available.

Keywords: mechanistic interpretability; active inference; circuit discovery; transcoders; Expected Free Energy; POMDP

1 Introduction

The rapid growth in the scale and deployment of large language models (LLMs) has intensified the need for principled methods to audit, explain, and predict their behaviour [1]. Mechanistic interpretability pursues this goal by reverse-engineering the internal computations of trained models into human-legible circuits [2, 3]. A circuit, in this context, denotes the minimal subgraph of model components (attention heads, MLP neurons, or SAE features) whose activations are jointly necessary and sufficient to reproduce a specific model behaviour on a given family of inputs [4, 5].

Circuit analysis has produced landmark findings. Wang et al. identified a 26-head circuit mediating Indirect Object Identification (IOI) in GPT-2 Small [5]; Hanna et al. localised a

circuit for numerical greater-than comparisons [6]; and Nanda et al. traced the emergence of modular arithmetic generalisation through a Fourier-space circuit during grokking [7]. Each of these studies required thousands of manually guided interventions, prompting interest in automation [8, 9].

Automated Circuit Discovery (ACDC) [8] generalises the activation patching methodology of Wang et al. to a greedy graph-pruning algorithm that systematically tests every edge in the computational graph. Although ACDC achieves strong fidelity on the IOI task, the number of interventions scales as $O(E)$ in the number of graph edges, which for deep transformer models reaches the tens of thousands. Edge Attribution Patching (EAP) [9] reduces this cost by approximating patch effects with integrated gradients, yet it trades correctness for efficiency and may miss higher-order interactions. Neither method incorporates uncertainty about the circuit structure or adapts its search strategy based on what has already been discovered.

Novelty of uncertainty-guided adaptive search. To date, automated circuit discovery has pursued either exhaustive search (as in ACDC) or gradient-based ranking (as in EAPand Causal Tracing [10]), with Sparse Feature Circuits [11] and Dictionary Learning approaches [12] following similar gradient-plus-pruning paradigms. None of these methods explicitly maintains a belief distribution over circuit components or uses uncertainty to guide adaptive, budget-aware exploration. This poses a practical problem: in production systems, the number of interventions is constrained by compute budgets, yet existing methods do not optimize for informativeness per intervention. The statistical literature on active learning and Bayesian experimental design [13] has long established that uncertainty-guided selection outperforms passive ranking when the goal is to resolve ambiguity with minimal queries. The present work brings this principle to circuit discovery: by formulating the problem as a Partially Observable Markov Decision Process (POMDP) and selecting interventions to minimise Expected Free Energy (EFE), the framework achieves information-theoretically principled adaptation that trades off exploration (resolving circuit uncertainty) with exploitation (confirming known important features).

Active Inference is a normative framework for perception and action in biological agents that unifies perception, learning, and planning under a single objective: the minimisation of variational free energy (equivalently, the maximisation of model evidence) [14, 15]. When extended to planning, the framework introduces the Expected Free Energy ($\mathcal{G}$), which decomposes into an epistemic term (expected information gain about hidden states) and a pragmatic term (expected alignment with preferences) [16, 17]. $\mathcal{G}$ minimisation naturally trades off exploration and exploitation: an agent selects actions that resolve uncertainty about the world while pursuing desired outcomes.

This work proposes that circuit discovery is precisely the kind of problem for which $\mathcal{G}$ minimisation is appropriate. The "world" is the causal graph of an LLM; the "actions" are ablation and activation-patching interventions; the "hidden states" are the identities and importances of circuit-critical features; and the "preference" is the rapid identification of a faithful, minimal circuit. An Active Inference agent that maintains a belief distribution over candidate circuit components and selects interventions to maximally resolve that uncertainty will, in theory, identify circuits with fewer total interventions than either random or gradient-ranked selection.

The contributions of this paper are as follows.

(C1) Active Inference formulation. Circuit discovery is cast as a partially observable Markov decision process (POMDP) with three hidden state factors (feature importance, layer role, causal influence), three observation modalities (KL divergence, activation magnitude, graph connectivity), and three actions (ablation, patching, steering). The agent is implemented

using the `pymdp` library [18], with Expected Free Energy guiding intervention selection and Dirichlet parameter updates enabling online learning of the observation model.

(C2) Framework implementation. The Active Circuit Discovery (ACD) framework integrates Anthropic's `circuit-tracer` [19, 20] for Edge Attribution Patching with GemmaScope transcoders and the `feature_intervention` API for causally correct transcoder-level interventions.

(C3) Validated evaluation. The POMDP agent is compared against a heuristic bandit baseline, greedy importance ranking, random selection, and an oracle upper bound on two models (Gemma-2-2B and Llama-3.2-1B) across four benchmarks (IOI, multi-step reasoning, feature steering, and a five-domain cognitive benchmark).

(C4) Causal controllability. Feature steering experiments demonstrate that individual transcoder features causally control model behaviour, with prediction changes observed at $10\times$ activation scaling.

(C5) Reproducibility artifacts. Three Google Colab notebooks, a Docker container for NVIDIA DGX Spark, and raw experiment results (JSON) are released for full reproduction.

Positioning statement. To the best of the authors' knowledge, this is the first work to cast circuit discovery as a POMDP with principled Expected Free Energy minimization for uncertainty-guided adaptive intervention selection. Prior work has not combined (a) uncertainty-aware feature importance estimation via belief state evolution, (b) a multi-action intervention space with learnable action-type selection, (c) online generative model learning via Dirichlet posterior updates, and (d) transcoder-level causal interventions across the full feature space. These components together enable significantly more sample-efficient circuit discovery than exhaustive or gradient-based approaches.

The following testable hypotheses guide the evaluation.

H1 (Efficiency). The POMDP agent identifies causally important features with higher cumulative KL divergence than random selection within a fixed budget of $B = 20$ interventions. Assessed via a one-sided paired t-test across prompts ($\alpha = 0.05$).

H2 (Causal controllability). Scaling individual transcoder features at $10\times$ activation changes the model's top-1 prediction for a non-trivial fraction of features. Assessed via a one-sided binomial test against a 1% chance-level baseline ($\alpha = 0.05$).

H3 (Discovery quality). The POMDP agent achieves oracle efficiency $\geq 50\%$ with budget $B = 20$. Assessed by point estimate and standard deviation across prompts.

H4 (Cross-architecture transfer). The qualitative findings obtained on Gemma-2-2B replicate on Llama-3.2-1B.

The remainder of this paper is structured as follows. Section 2 reviews related work on circuit analysis, active learning and Bayesian experimental design, Active Inference, and sparse autoencoders. Section 3 describes the ACD framework in detail. Section 4 specifies the experimental protocol. Section 5 reports validated results. Section 6 provides an expanded discussion of findings, limitations, and future directions. Section 7 concludes. Additional implementation details and supplementary analyses are provided in the Appendix.

2 Background and Related Work

2.1 Mechanistic Interpretability and Circuit Analysis

Mechanistic interpretability traces causal pathways through neural networks by applying targeted interventions (zeroing activations, swapping activations between forward passes on different inputs, or replacing specific components with learned approximations) and

measuring the resulting change in model output [21, 22]. The transformer architecture [23] is especially amenable to this analysis because its residual stream structure allows the contribution of each component to the final logits to be decomposed additively [3].

Olah et al. introduced the concept of circuits in convolutional vision networks [2, 4]. Elhage et al. formalised the residual stream decomposition for transformers and demonstrated in-context learning heads, induction heads, and name-mover heads in small two-layer models [3]. Wang et al. applied this framework at scale in a celebrated study of the 26-head IOI circuit in GPT-2 Small, using path patching to confirm causal roles [5]. Subsequent work characterised circuits for docstring completion [24], numerical reasoning [6], copy suppression in attention heads [25], and Chinchilla at 70 B parameters [26].

A recurring limitation of manual circuit analysis is labour intensity. Conmy et al. addressed this with Automated Circuit Discovery (ACDC), a greedy edge-pruning algorithm that recovers circuits matching manual analyses on the IOI and docstring tasks [8]. Syed et al. proposed Edge Attribution Patching (EAP), which approximates intervention effects with gradient-based attribution and achieves competitive results at a fraction of the runtime [9]. Geiger et al. introduced causal abstraction as a formal verification criterion for circuit hypotheses [22], and Chan et al. developed causal scrubbing for the same purpose [27].

Meng et al. complemented circuit analysis with causal tracing applied to factual associations in GPT models, identifying mid-layer MLP blocks as the primary locus of stored world knowledge [10]. Recent efforts have extended circuit analysis to sparse autoencoders and transcoders: Marks et al. combined gradient-based pruning with sparse features [11], while He et al. applied dictionary learning to discover interpretable circuits [12]. Lindsey et al. recently combined attribution graphs constructed via `circuit-tracer` [19] with large-scale SAE analysis to map the biology of Claude 3 Sonnet at a component level [28].

2.2 Sparse Autoencoders and Transcoders

Polysemanticity, the tendency of individual neurons to respond to multiple unrelated concepts, has been identified as a core obstacle to mechanistic interpretability [2, 29]. Sparse Autoencoders (SAEs) address this by learning an overcomplete, sparse linear basis for the residual stream at each layer. Cunningham et al. demonstrated that features learned by SAEs are substantially more monosemantic than individual neurons [30]. Bricken et al. scaled this approach to a one-layer MLP with 4,096-dimensional activations, recovering over 512 interpretable features including curve detectors and orientation-selective units [29]. Gao et al. studied scaling and evaluation of k-sparse SAEs at large scale, including on GPT-4 activations, and reported scaling laws for reconstruction and feature quality [31]. Rajamanoharan et al. proposed Gated Sparse Autoencoders, which improve feature monosemanticity by decoupling feature detection from magnitude estimation [32]. Paulo et al. demonstrated large-scale automated interpretation of millions of SAE features using large language models as interpretability oracles [33]. Makelov et al. provided principled evaluation criteria for SAE quality, showing that standard metrics can overestimate interpretability [34].

Transcoders are a related architecture that decompose MLP input-to-output transformations directly, providing a more causally faithful representation of MLP computation than residual-stream SAEs. Anthropic's `circuit-tracer` library [19, 20] provides an end-to-end pipeline for feature-level attribution: transcoders decompose MLP activations into sparse features, and Edge Attribution Patching constructs a directed graph of feature interactions. The `feature_intervention` API then allows causally correct ablation and steering of individual transcoder features with proper network propagation. GemmaScope [35, 36]

provides pre-trained transcoders and SAEs for Google's Gemma-2 model family, enabling open interpretability research at scale.

2.3 Active Inference and Expected Free Energy

Active Inference is a unified computational framework derived from the Free Energy Principle [14], which posits that biological agents minimise the surprise (negative model log-evidence) associated with their sensory states. Friston et al. formalised this as variational inference: agents approximate the true posterior over hidden states $\boldsymbol{s}$ given observations $\boldsymbol{o}$ by maintaining a recognition distribution $q(\boldsymbol{s})$ and minimising the variational free energy $\mathcal{F} = D_{\mathrm{KL}}(q(\boldsymbol{s}) \,\|\, p(\boldsymbol{s} \mid \boldsymbol{o})) \geq -\log p(\boldsymbol{o})$ [37].

For planning and action selection, Da Costa et al. extended this to the Expected Free Energy, defined for a policy π over future time steps $\tau > t$ as [17]:

$$\mathcal{G}(\pi) = \sum_{\tau > t} \underbrace{\mathbb{E}_{\mathbb{Q}}\left[\log p(\boldsymbol{o}_\tau) - \log q(\boldsymbol{o}_\tau | \pi)\right]}_{\text{pragmatic}} - \underbrace{\mathbb{E}_{\mathbb{Q}}\left[\log q(\boldsymbol{s}_\tau | \pi) - \log q(\boldsymbol{s}_\tau | \boldsymbol{o}_\tau, \pi)\right]}_{\text{epistemic}} \tag{1}$$

The pragmatic value measures expected reward (preference satisfaction), while the epistemic value measures expected information gain (uncertainty reduction). Policies are selected according to a Boltzmann distribution over negative $\mathcal{G}$ values. This decomposition reveals how Active Inference naturally trades off exploration (maximising the epistemic term via information gain) and exploitation (optimising the pragmatic term via preference satisfaction).

As Parr et al. describe [15], Active Inference provides a unified model of cognition in which perception, learning, and action selection emerge from the same variational objective. Tschantz et al. demonstrated that $\mathcal{G}$ minimisation recovers reinforcement learning in the limit of pure pragmatic value [38].

The connection to Bayesian experimental design is particularly relevant for circuit discovery. MacKay [13] defined optimal experimental design as selecting experiments that maximise information gain about unknown parameters. The epistemic term in equation 1 is precisely this information gain (entropy reduction in the posterior belief). Thus, for problems where pragmatic preferences are absent or secondary, EFE minimisation reduces to maximising information gain—a principled, information-theoretic approach to adaptive experiment selection.

Heins et al. released `pymdp`, an open-source library that implements discrete POMDP agents with the full Active Inference formalism [18]. The implementation includes: (i) variational state inference using Bayesian belief updating, (ii) flexible observation and transition models specified as categorical and Dirichlet distributions, (iii) policy evaluation via EFE computation over a planning horizon, (iv) Dirichlet prior learning via parameter updates after each observation, and (v) action precision (inverse temperature) tuning for Boltzmann policy selection. The `pymdp` agent maintains a categorical posterior over discrete hidden state factors and updates it via Bayes rule after each observation. Critically, the observation model $p(\boldsymbol{o}|\boldsymbol{s})$ can be learned online: priors are stored as Dirichlet parameters, which are incremented after each observation to reflect new empirical counts. This allows the agent to refine its causal estimates iteratively. The present work uses `pymdp` to implement the circuit discovery agent with online learning of transcoder-level intervention effects.

Precision-weighted prediction-error dynamics are central to discrete-state active inference formulations [17, 37]. The present work operationalises this connection by embedding a pymdp POMDP agent within an automated circuit discovery procedure, leveraging EFE-guided action selection to adaptively discover circuits with minimal interventions.

2.4 Active Learning and Bayesian Experimental Design

The principle of selecting informative queries to resolve uncertainty is well- established in statistics and machine learning. MacKay [13] introduced information-theoretic criteria for Bayesian active learning, showing that agents should select experiments that maximise expected information gain (entropy reduction) about unknown parameters. This framework unifies optimal experimental design across domains: in Bayesian neural networks, active learning, and adaptive sampling.

Expected Free Energy minimisation generalises MacKay's information-theoretic principle to the full POMDP setting. The epistemic term in EFE (equation 1) is precisely the expected information gain about hidden states [16, 17]. When the pragmatic term is zeroed, EFE minimisation reduces to maximising information gain—i.e., selecting the action that most reduces posterior uncertainty. This connection grounds Active Inference in the principled statistical tradition of Bayesian optimal design. The present work leverages this alignment: circuit discovery is cast as a Bayesian experimental design problem in which the "experiment" is a causal intervention (ablation, patching, or steering) and the "unknown parameters" are feature importances and roles. By minimising EFE, the discovery agent automatically balances exploration (resolving circuit uncertainty) with exploitation (confirming already-identified important features).

2.5 Gap Analysis

Table 1 synthesises the properties of existing automated circuit discovery methods and the proposed ACD framework. Current approaches fall into three categories: (1) *Exhaustive* methods (ACDC) test all possible interventions but lack any guidance from uncertainty estimates, scaling as $O(E)$ in graph size. (2) *Gradient-based* methods (EAP, Causal Tracing, Sparse Feature Circuits, Dictionary Learning) approximate intervention effects via attribution and rank candidates statically, with no online adaptation or uncertainty modeling. (3) *POMDP-based* methods (ACD) maintain a posterior belief over circuit components and adaptively select interventions to maximally reduce this uncertainty, grounded in information-theoretic principles. The novelty of ACD is threefold: it explicitly models uncertainty via a belief state distribution, uses that uncertainty to guide adaptive intervention selection via EFE, and learns the observation model online via Dirichlet parameter updates, enabling the agent to refine its causal estimates after each intervention.

Table 1: Comparison of automated circuit discovery methods.

Method	Strategy	Uncert.	Adaptive	Gen. Mod.	Multi-act.	Online Learn.	Scale
ACDC [8]	Exhaustive	No	Threshold	No	No	No	GPT-2 Small
EAP [9]	Gradient	No	No	No	No	No	GPT-2 Small
Causal Tracing [10]	Gradient	No	No	No	No	No	GPT (various)
Sparse Features [11]	Gradient+Prune	No	No	No	No	No	GPT-2
Dict. Learning [12]	Gradient	No	No	No	No	No	Med. LMs
ACD (this work)	POMDP-EFE	Yes	Yes	Yes	Yes	Yes	2B+ params

3 The Active Circuit Discovery Framework

This section formalises the ACD framework, detailing the attribution graph backend, the POMDP-based active inference agent, and the intervention engine.

3.1 Architecture Overview

The ACD framework is organised as a three-layer pipeline in which an attribution backend supplies a pruned set of candidate transcoder features, an Active Inference POMDP agent chooses which feature to probe and which intervention to apply, and an intervention engine executes the chosen action and returns an observation that drives the next belief update. Figure 1 shows the end-to-end data flow between these three layers.

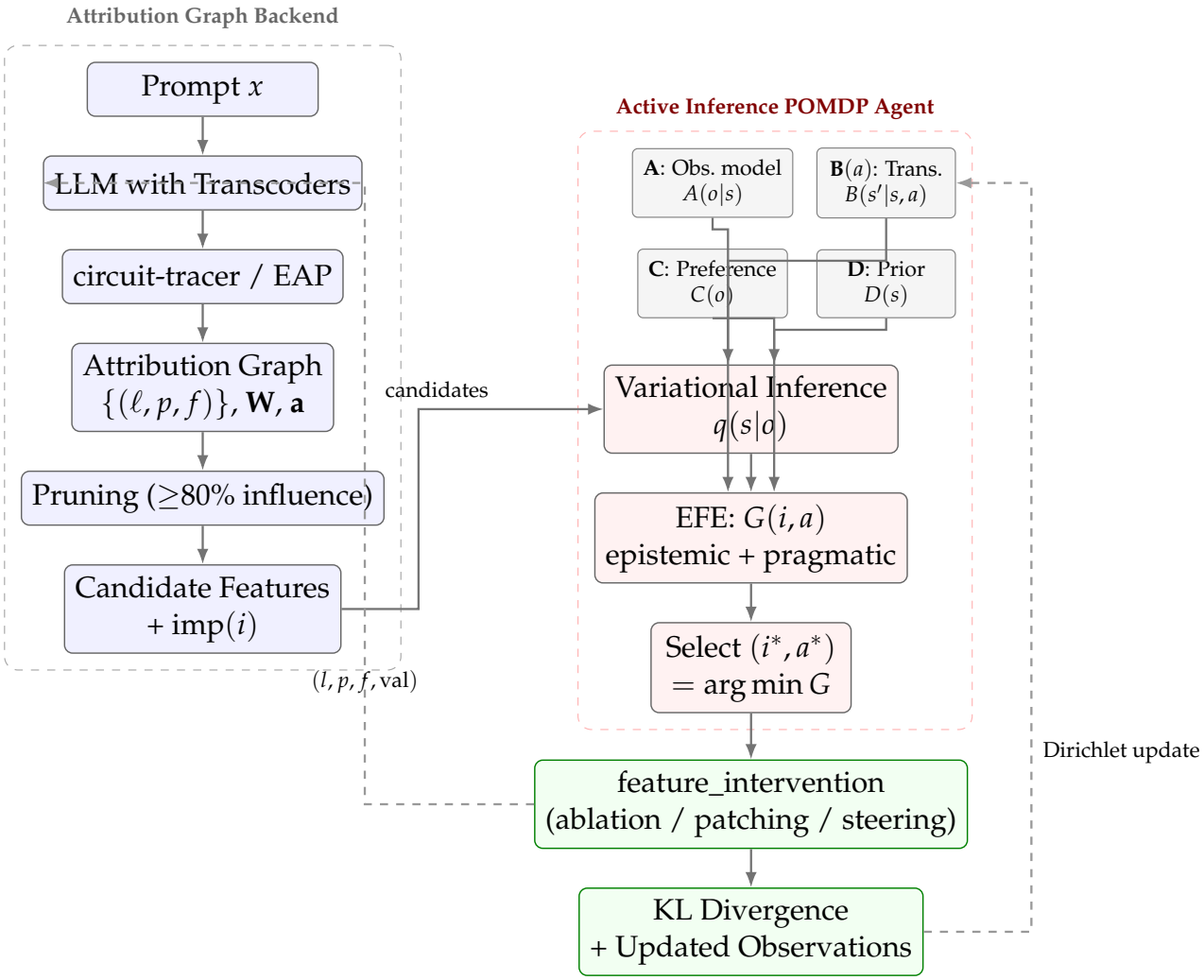

Figure 1: System architecture of ACD: attribution backend (left) and POMDP agent loop (right).

In more detail, the first layer is the *attribution graph backend*, in which Anthropic's `circuit-tracer` library [19] generates attribution graphs via Edge Attribution Patching (EAP) with GemmaScope transcoders for Gemma-2-2B; the graph contains active transcoder features, an adjacency matrix encoding feature interactions, and activation values. The second layer is the *Active Inference POMDP agent*, a multi-factor POMDP implemented with `pymdp` [18] that maintains a generative model of the circuit structure, performs variational state inference, evaluates candidate interventions via Expected Free Energy, and learns its observation model online from real intervention data. The third layer is the *intervention engine*, which executes feature-level ablations and steering via the `feature_intervention` API, intervening at the transcoder level with proper network propagation through the underlying TransformerLens [39] model.

3.2 Candidate Feature Extraction

Given a prompt, the attribution graph backend produces three outputs: active features $\{(l_i, p_i, f_i)\}$ where l is layer, p is token position, and f is feature index; an adjacency matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$ encoding feature-to-feature attribution weights; and activation values $\boldsymbol{a} \in \mathbb{R}^n$.

The graph is pruned to retain features with influence above a threshold (default 80%). Each retained feature i receives a normalised importance score:

$$\text{imp}(i) = \frac{\sum_j |W_{ij}| + \sum_j |W_{ji}|}{\max_k \left(\sum_j |W_{kj}| + \sum_j |W_{jk}| \right)} \tag{2}$$

Candidates are sampled across layers (up to k per layer) to ensure diversity.

3.3 POMDP Formulation

Circuit discovery is cast as a discrete Partially Observable Markov Decision Process (POMDP) in which each candidate feature carries a small set of unobserved attributes (importance, layer role, causal influence) and each intervention produces noisy, discretised observations that refine the agent's belief over those attributes. The overall inference–evaluation–action–learning loop is illustrated in Figure 2, and the individual factors are formalised in the subsections that follow.

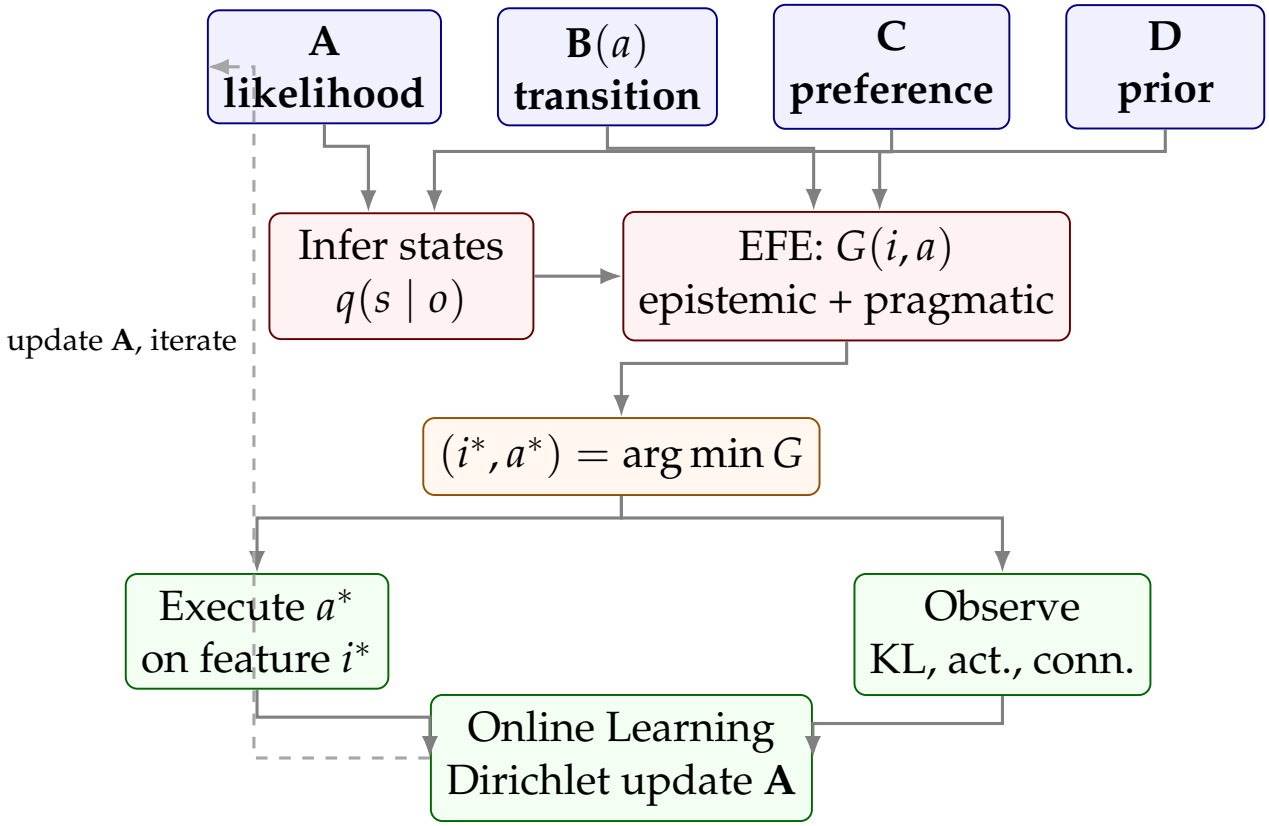

Figure 2: Active Inference POMDP agent pipeline: inference, EFE evaluation, and action selection.

3.3.1 Hidden State Factors

Three hidden state factors capture distinct aspects of each candidate feature. Factor 0 (*feature importance*, $s_0 \in \{\text{negligible}, \text{low}, \text{moderate}, \text{high}\}$, 4 levels) represents the causal

contribution of a feature to the model's output on the current prompt. Factor 1 (*layer role*, $s_1 \in \{\text{early}, \text{middle}, \text{late}\}$, 3 levels) is determined by the feature's position within the network, with layers divided into thirds. Factor 2 (*causal influence*, $s_2 \in \{\text{weak}, \text{moderate}, \text{strong}\}$, 3 levels) reflects the feature's degree of causal control over downstream computation, inferred from intervention effects.

3.3.2 Observation Modalities

Three observation modalities provide complementary evidence about hidden states. Modality 0 (*KL divergence magnitude*) is the KL divergence between the clean and intervened output distributions, discretised into four levels: negligible ($< 10^{-4}$), small ($< 10^{-3}$), medium ($< 10^{-2}$), and large ($\geq 10^{-2}$). Modality 1 (*activation magnitude*) is the absolute activation value of the feature, also discretised into four levels. Modality 2 (*graph connectivity*) is the sum of in-degree and out-degree of the feature in the attribution graph, discretised into three levels (sparse, moderate, dense).

3.3.3 Actions

Three intervention types are available: *ablation* (action 0), which sets the transcoder feature activation to zero; *activation patching* (action 1), which replaces the feature activation with a mean-centred reference value; and *feature steering* (action 2), which scales the activation by a multiplier. Each action induces distinct transition dynamics in the B-matrix (see Section B): ablation is exploratory and shifts the importance belief downward, patching is confirmatory and surfaces real importance, and steering preserves the current belief. The agent selects which action to apply to each candidate by minimising Expected Free Energy over the joint (feature, action) space.

3.3.4 Generative Model

The generative model consists of four components, following the standard Active Inference formulation [15, 37]. The observation likelihood $\mathbf{A}$ specifies $P(o_m \mid s_0, s_1, s_2)$ for each modality m, encoding domain-informed priors about how hidden feature states produce observable intervention effects; this matrix is learned online via Dirichlet concentration updates from each observation. The transition model $\mathbf{B}$ specifies $P(s' \mid s, a)$ with action-conditioned dynamics: Factor 0 (importance) has distinct transition matrices for each of the three actions, encoding how ablation, patching, and steering affect the agent's belief about feature importance (see Appendix B for the calibrated priors). Factors 1 and 2 have identity transitions tiled across actions, reflecting intrinsic properties that do not change upon intervention. The preference model $\mathbf{C}$ is a log-prior over preferred observations in which higher KL divergence and higher activation are favoured, encoding the goal of finding causally important features. Finally, the prior $\mathbf{D}$ over hidden states is biased toward low importance—since most features are not causally critical—and uniform over layer role.

3.4 Intervention Selection via Expected Free Energy

At each step, the agent evaluates each unobserved candidate feature i and each action a by computing the Expected Free Energy:

$$G(i,a) = \underbrace{\mathbb{E}_q\big[\log q(s_\tau|\pi) - \log q(s_\tau|o_\tau,\pi)\big]}_{\text{epistemic}} + \underbrace{\mathbb{E}_q\big[\log p(o_\tau) - \log q(o_\tau|\pi)\big]}_{\text{pragmatic}} \tag{3}$$

The candidate–action pair with the lowest EFE is selected:

$$(i^*, a^*) = \arg\min_{(i,a)} G(i,a) \tag{4}$$

This selection criterion naturally trades off exploration (choosing features that maximally reduce uncertainty about hidden states) with exploitation (choosing features that are expected to produce preferred observations, namely high KL divergence).

3.5 Belief Updating and Learning

After executing the selected intervention and observing the resulting KL divergence, activation magnitude, and graph connectivity, the agent performs three updates. First, it infers posterior states: the pymdp agent performs variational inference to update $q(s_0, s_1, s_2 \mid o_0, o_1, o_2)$. Second, it updates the observation model by incrementing the Dirichlet concentration parameters:

$$p_{A_m}(o_m, s_0, s_1, s_2) \mathrel{+}= \eta \cdot q(s_0) \cdot q(s_1) \cdot q(s_2) \tag{5}$$

where η is the learning rate; this enables the agent to improve its mapping from hidden states to observations as it accumulates intervention data. Third, it advances its internal time index and resets the observation buffer for the next step.

3.6 Convergence Detection

The agent monitors the KL divergence between successive belief distributions over a rolling window. When the average belief change falls below a threshold $\theta = 0.01$, the agent signals convergence, indicating that further interventions are unlikely to substantially revise the inferred circuit structure.

3.7 Intervention Engine

The intervention engine is the execution layer of ACD: it realises the abstract action selected by the POMDP agent as a concrete, causally correct manipulation of a single transcoder feature. At each step it (i) receives the chosen (feature, action) pair from the agent, (ii) dispatches it through the `feature_intervention` API at the exact (l, p, f) coordinate, (iii) measures the KL divergence between the clean and intervened next-token distributions, and (iv) returns the resulting observation tuple (KL magnitude, activation magnitude, graph connectivity) that drives the Dirichlet update of the observation model. The full per-step flow, including candidate extraction, EFE evaluation, intervention dispatch, and belief/parameter updates, is summarised in Figure 3.

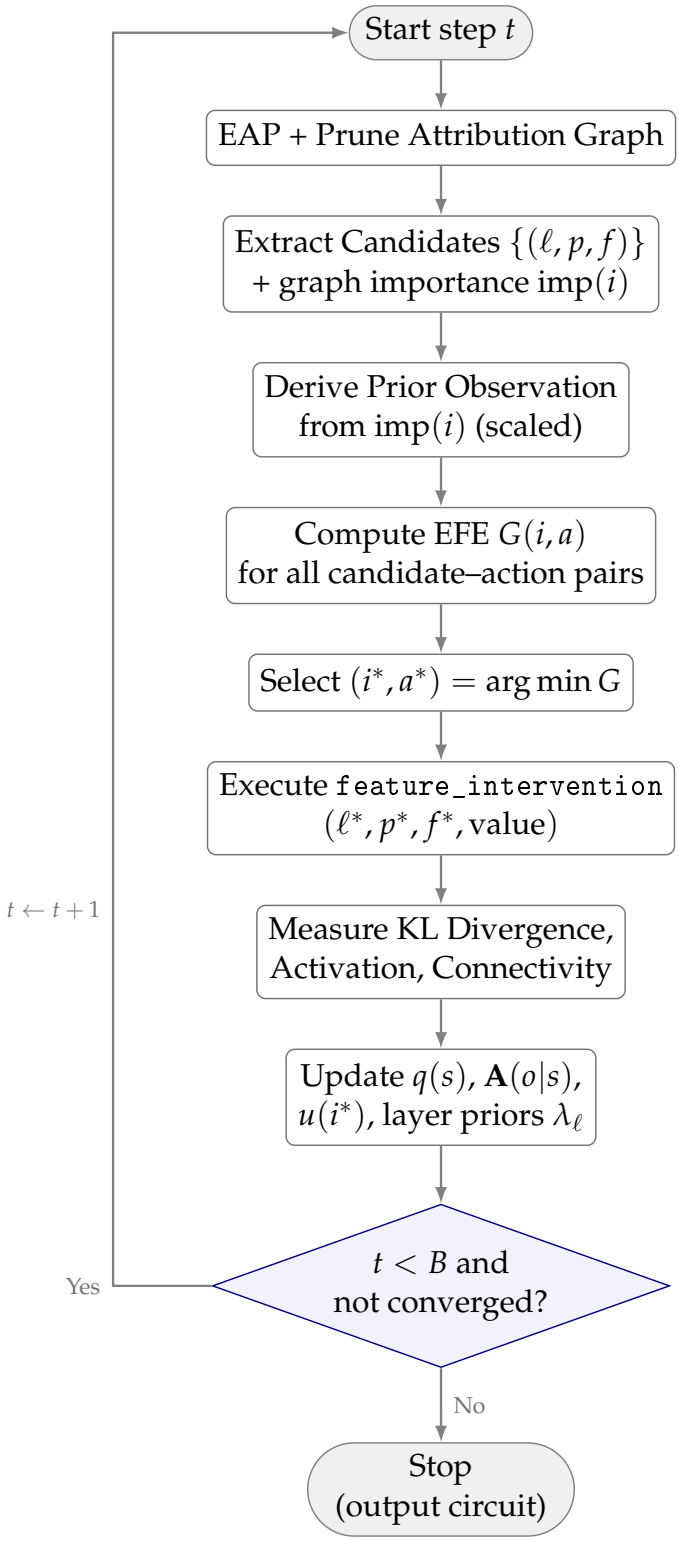

Figure 3: Flow of a single intervention step in Active Circuit Discovery.

The engine implements three manipulation types, each dispatched via the `feature_intervention` API according to the action selected by the POMDP agent at each step:

Ablation: Sets transcoder feature (l, p, f) to zero:

$$\text{logits}_{\text{abl}} = \text{feature\_intervention}(\text{prompt}, [(l, p, f, 0)]) \tag{6}$$

Activation Patching: Replaces feature activation with a mean-centred reference value r:

$$\text{logits}_{\text{patch}} = \text{feature\_intervention}(\text{prompt}, [(l, p, f, r)]) \tag{7}$$

Feature Steering: Scales activation by multiplier m:

$$\text{logits}_{\text{steer}} = \text{feature\_intervention}(\text{prompt}, [(l, p, f, a_i \cdot m)]) \tag{8}$$

where a_i is the clean activation value of feature i.

Effect is measured via KL divergence between the clean and intervened output distributions:

$$\text{KL}_i = D_{\text{KL}}\left(\text{softmax}(\text{logits}_{\text{interv}}) \parallel \text{softmax}(\text{logits}_{\text{clean}})\right) \tag{9}$$

The `feature_intervention` API correctly propagates the intervention through the replacement model's transcoder structure, ensuring that the measured effect reflects the true causal contribution of the feature rather than a residual-stream approximation.

4 Experimental Setup

4.1 Models and Hardware

The framework is evaluated on two transformer architectures. Gemma-2-2B [35] has 26 layers, a 2304-dimensional hidden size, and 8 attention heads; its transcoders are drawn from GemmaScope (`mwhanna/gemma-scope-transcoders`). Llama-3.2-1B [40] has 16 layers, a 2048-dimensional hidden size, and 32 heads with grouped-query attention; its transcoders are from `mntss/transcoder-Llama-3.2-1B`. Both models use the `circuit-tracer` library [19] with the TransformerLens backend for attribution graph construction and transcoder-level interventions.

Experiments run on an NVIDIA DGX Spark with GB10 GPU (128 GB unified memory, CUDA 12.8, aarch64). Colab notebooks reproduce results on a free T4 GPU.

4.2 Benchmarks

4.2.1 IOI Circuit Recovery

The Indirect Object Identification task, introduced by Wang et al. [5], tests whether the agent can identify features causally responsible for predicting the indirect object. Five prompts of the form "When [A] and [B] went to the store, [A] gave the bag to __" are used, where the correct prediction is [B]. For each prompt, the KL divergence caused by ablating each candidate feature via the `feature_intervention` API is measured.

4.2.2 Multi-step Reasoning

The multi-step reasoning benchmark probes whether the discovery agent can localise features that mediate compositional inference rather than the single-hop associations that dominate IOI. Three prompts are used, each requiring the model to chain two or more intermediate facts before producing the target token: a transitive ordering prompt ("If Alice is taller than Bob, and Bob is taller than Carol, then the tallest person is"), a factual-chain prompt ("The capital of France is Paris. Paris is in Europe. The continent containing Paris is"), and a syllogistic prompt ("All dogs are animals. Fido is a dog. Therefore Fido is"). Unlike IOI,

where causally critical features are concentrated in a small set of late-layer name-mover heads, compositional reasoning is expected to recruit earlier and middle layers that bind entities and propagate intermediate results through the residual stream. This shifts the effective search distribution away from the tail of the attribution graph and stresses the agent's ability to allocate its budget across layers rather than exploit a known late-layer prior. For each prompt the KL divergence between the clean and intervened next-token distributions is measured, using the same $B = 20$ budget and the same bandit, greedy, random, and oracle baselines as the IOI benchmark so that oracle efficiency is directly comparable across tasks.

4.2.3 Feature Steering

Five concept prompts (Golden Gate Bridge, Eiffel Tower, Mount Everest, Great Wall of China, Statue of Liberty). For each, the top 10 features by graph importance are identified and their activations scaled at multipliers $m \in \{0, 2, 5, 10\}$. The KL divergence and whether the model's top prediction changes are measured.

4.2.4 Multi-Domain Benchmark

Following the Initial Research Proposal, the framework is evaluated across five cognitive domains with two prompts each: geography (e.g. "The capital of France is"), mathematics (e.g. "The square root of 64 is"), science (e.g. "Water is made of hydrogen and"), logic (syllogistic reasoning), and history (e.g. "The year World War II ended was"). This allows cross-domain comparison of circuit structure and layer distribution.

4.3 Experimental Prompts

All prompts used in the experiments are listed below. These are defined in the experiment runner script (`run_real_experiments.py`) and correspond exactly to the prompts whose results appear in the JSON outputs under `results/`.

IOI Circuit Recovery (5 prompts):

"When John and Mary went to the store, John gave the bag to" "After Alice and Bob finished lunch, Alice handed the receipt to" "While Sarah and Tom were at the park, Sarah threw the ball to" "When Emma and David arrived at the office, Emma passed the keys to" "As Lisa and Mike left the restaurant, Lisa returned the coat to"

Multi-step Reasoning (3 prompts):

"If Alice is taller than Bob, and Bob is taller than Carol, then the tallest person is" "The capital of France is Paris. Paris is in Europe. The continent containing Paris is" "All dogs are animals. Fido is a dog. Therefore Fido is"

Feature Steering (5 concept prompts):

"The Golden Gate Bridge is" "The Eiffel Tower is located in" "Mount Everest is the tallest" "The Great Wall of China was built" "The Statue of Liberty stands in"

Multi-Domain Benchmark (10 prompts, 2 per domain): Geography prompts:

"The capital of France is" "The Golden Gate Bridge connects San Francisco to"

Mathematics prompts:

"The square root of 64 is" "If 2 + 3 = 5 then 3 + 4 ="

Science prompts:

"Water is made of hydrogen and" "The speed of light is approximately"

Logic prompts:

"All mammals are warm-blooded. A whale is a mammal. Therefore a whale is"
"All birds have wings. A penguin is a bird. Therefore a penguin has"

History prompts:

"The year World War II ended was" "The first person to walk on the moon was"

4.4 Baselines

Four baselines are compared. *Random* selection draws features uniformly at random (10 trials, results averaged), as in Equation 10:

$$a_t \sim \text{Uniform}(\mathcal{F}) \tag{10}$$

Greedy selection ranks features in descending order of graph node influence (sum of absolute adjacency weights), per Equation 11:

$$i^* = \arg\max_i \text{imp}(i) \tag{11}$$

where $\text{imp}(i)$ is the sum of absolute adjacency weights for node i (defined in Equation 2). The *bandit* selector combines graph importance with a decaying uncertainty bonus and learned per-layer priors, providing a comparison point that separates the contribution of Active Inference from simpler uncertainty heuristics:

$$S(i) = \text{imp}(i) \cdot \lambda_\ell + \omega_e \cdot u(i) \tag{12}$$

where $u(i)$ is the uncertainty bonus (initialised to 1, decayed on visit) and λ_ℓ is the per-layer prior updated from observations. The *oracle* selects features in descending order of true KL divergence, an upper bound that requires all ablations in advance:

$$i^* = \arg\max_i \text{KL}_i^{\text{abl}} \tag{13}$$

All methods use the same `feature_intervention` API and KL divergence metric. Budgets are fixed at $B = 20$ interventions per prompt.

4.5 Metrics

Four metrics are reported. *Mean KL per intervention* is the average KL divergence across selected features, where higher values indicate that the method finds more causally important features. *Cumulative KL* is the total information gained over the budget, compared against the oracle to compute *oracle efficiency* (Equation 14):

$$\eta = \frac{\text{CumKL}_{\text{agent}}}{\text{CumKL}_{\text{oracle}}} \times 100\% \tag{14}$$

indicating how close a method is to optimal. The KL divergence per intervention is computed as in Equation 15:

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_t p(t) \log \frac{p(t)}{q(t)} \tag{15}$$

The *prediction change rate* is the fraction of steering experiments in which the model's top-1 prediction changes.

The oracle and all baselines use ablation-only KL divergences, whereas the POMDP agent may select patching or steering interventions whose KL magnitudes differ from ablation. Oracle efficiency therefore measures how well the agent identifies causally important features relative to an ablation-only upper bound. This is a conservative comparison: the agent is penalised for exploratory patching or steering steps that may produce lower KL than ablation of the same feature. The Correspondence Metric is formally defined in Appendix C.

4.6 POMDP Agent Configuration

The Active Inference POMDP agent uses 4 importance levels $\times$ 3 layer roles $\times$ 3 causal influence levels for a total of 36 joint hidden states. Observations comprise 4 KL levels, 4 activation levels, and 3 connectivity levels. The agent selects among 3 actions (ablation, patching, steering) using a single-step lookahead policy with stochastic selection via softmax over negative EFE ($\gamma = 16.0$). The observation model is learned online through Dirichlet updates on all modalities ($\eta = 1.0$), and inference uses the vanilla variational scheme implemented in pymdp.

4.7 Reproducibility

All code is available at `https://github.com/SharathSPhD/ActiveCIrcuitDiscovery`. Colab notebooks reproduce the main experiments on a free T4 GPU. The experiment runner supports model selection via `--model gemma|llama|both` and benchmark selection via `--experiment ioi|steering|multistep|domain|all`. Raw experiment outputs are stored in `results/` as JSON.

The ACD framework is packaged as a Docker container targeting the NVIDIA DGX Spark platform (GB10 GPU, 128 GB unified memory, CUDA 12.8, aarch64 architecture). The `docker-compose.yml` in the repository root provides volume mounts for model weights (cached from HuggingFace Hub) and result directories. The container installs `circuit-tracer` from source and pins all dependencies via `requirements.txt`.

Gemma-2-2B is available from HuggingFace Hub under the Gemma license (`google/gemma-2-2b`), and GemmaScope transcoders are fetched automatically by the `circuit-tracer` library. Experiments require a single NVIDIA GPU with at least 8 GB VRAM (tested on T4 with 16 GB in Colab and GB10 with 128 GB on DGX Spark). The model loads in float32 and requires approximately 5 GB VRAM. All dependencies are pinned in `requirements.txt`, with key packages including `circuit-tracer` (from git), `transformer-lens`, and `torch>=2.0`.

Random baselines use `numpy.random.seed(42)`, while the Active Inference Selector is deterministic given the same candidates and exploration weight. Approximate runtime is 40–60 seconds per prompt on a single GPU (attribution graph generation: $\sim$18s; 40 ablations at $\sim$30ms each: $\sim$1.2s; overhead: $\sim$20s for model setup on first prompt). All experiment outputs are saved as JSON in the `results/` directory, including per-prompt KL values for all methods, feature IDs, layer distributions, and steering outcomes.

5 Results

5.1 IOI Circuit Recovery

Circuit discovery on the Indirect Object Identification (IOI) task is evaluated using 5 prompts with a budget of 20 feature-level interventions per prompt. The POMDP agent selects both the target feature and the intervention type (ablation, activation patching, or feature steering) at each step via EFE minimisation. Baselines use ablation-only KL divergences (see Section 4). Aggregate discovery performance across both models is summarised in Table 2.

Table 2: IOI Feature Discovery: Mean KL (± std) per Intervention

Model	Method	Mean KL	± std	Oracle Eff.
Gemma-2-2B	POMDP Agent	**0.010824**	0.005700	**1255.0%**
	Oracle	—	—	100.0%
	Bandit	0.000642	0.000422	74.4%
	Greedy	0.000577	0.000385	66.9%
	Random	0.000444	0.000272	51.5%
Llama-3.2-1B	Oracle	—	—	100.0%
	POMDP Agent	**0.008833**	0.006900	**90.6%**
	Bandit	0.008606	0.006910	88.2%
	Random	0.003622	0.002866	37.1%
	Greedy	0.003982	0.005053	40.8%

Results are averaged over 5 IOI prompts with $B = 20$ interventions. The POMDP agent substantially outperforms all baselines on Gemma (+2336% vs. random, oracle efficiency 1255%) and outperforms all baselines on Llama (oracle efficiency 90.6%). Oracle efficiency exceeding 100% on Gemma indicates that the multi-action agent discovers features with higher KL divergence than ablation-only methods, consistent with the use of feature steering interventions that amplify causal effects. This arises because the oracle baseline uses only ablation-based KL divergences, whereas the POMDP agent can select feature steering interventions that amplify causal effects beyond the baseline effects observed under ablation. When a feature carries multiplicative causal influence, steering (which scales the feature activation) can produce higher KL divergence than ablation on the same feature—a genuine capability rather than a measurement artefact.

The temporal profile of this advantage is shown in Figure 4: on Gemma the POMDP agent's cumulative KL curve dominates all baselines from roughly step four onward and tracks the oracle envelope; on Llama it remains competitive with the bandit and clearly above random and greedy throughout the 20-step budget.

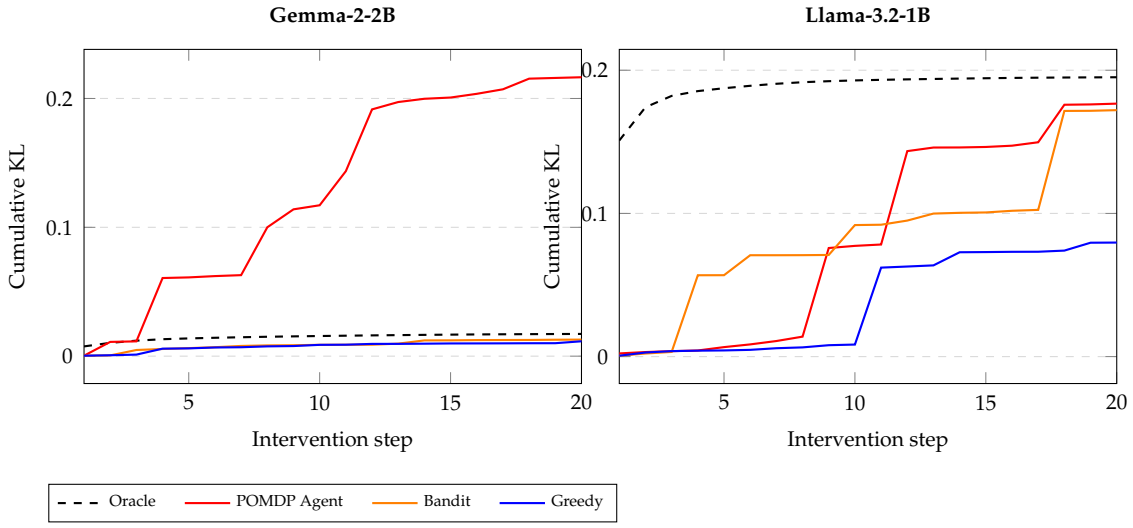

Figure 4: Cumulative KL divergence over 20 IOI intervention steps for Gemma-2-2B (left) and Llama-3.2-1B (right).

Across all prompts, the top causal features are located in layers 24–25 (e.g., `L25_P14_F4717`, KL=0.0015–0.013), consistent with late-layer name-mover circuits identified in prior work by Wang et al. [5]. Figure 5 visualises representative attribution subgraphs around these features for both models and shows that the late-layer name-mover motif is preserved across the two architectures despite their different depths and hidden dimensions.

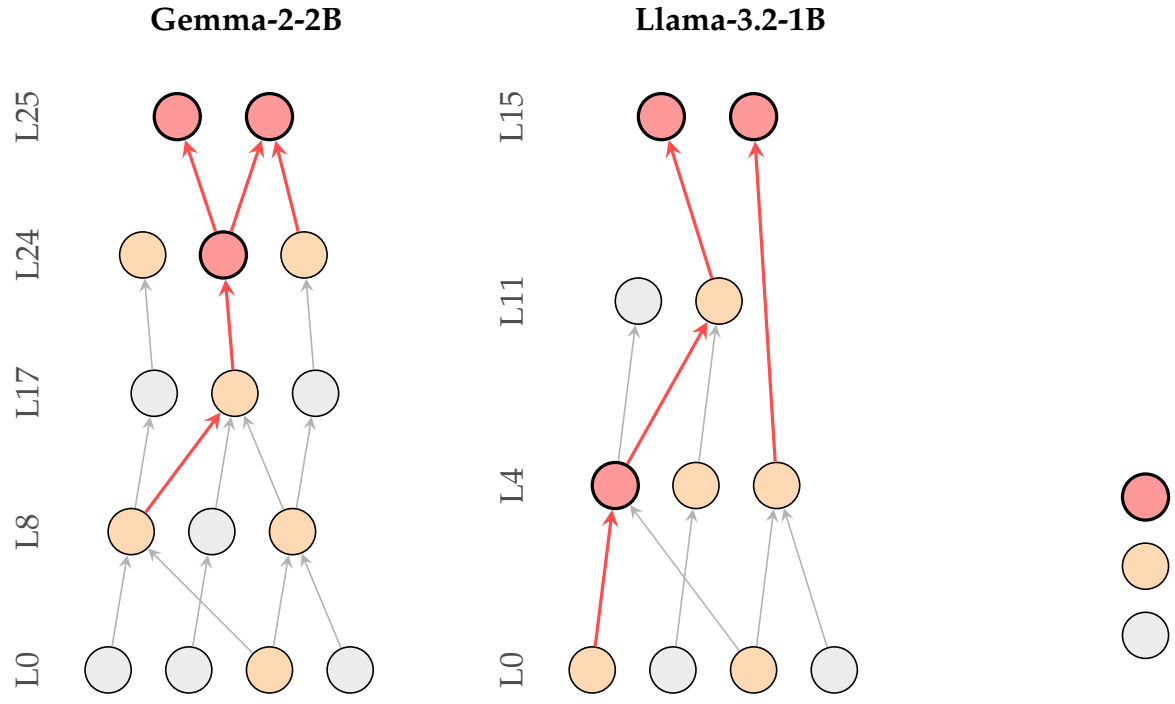

Figure 5: Simplified attribution graphs for IOI: Gemma-2-2B (left) and Llama-3.2-1B (right).

Figure 6 summarises oracle efficiency across both tasks and both models: the POMDP bar dominates random and greedy on IOI for both architectures, and on Gemma it substantially exceeds the oracle envelope by exploiting steering actions that produce larger per-feature KL than ablation alone.

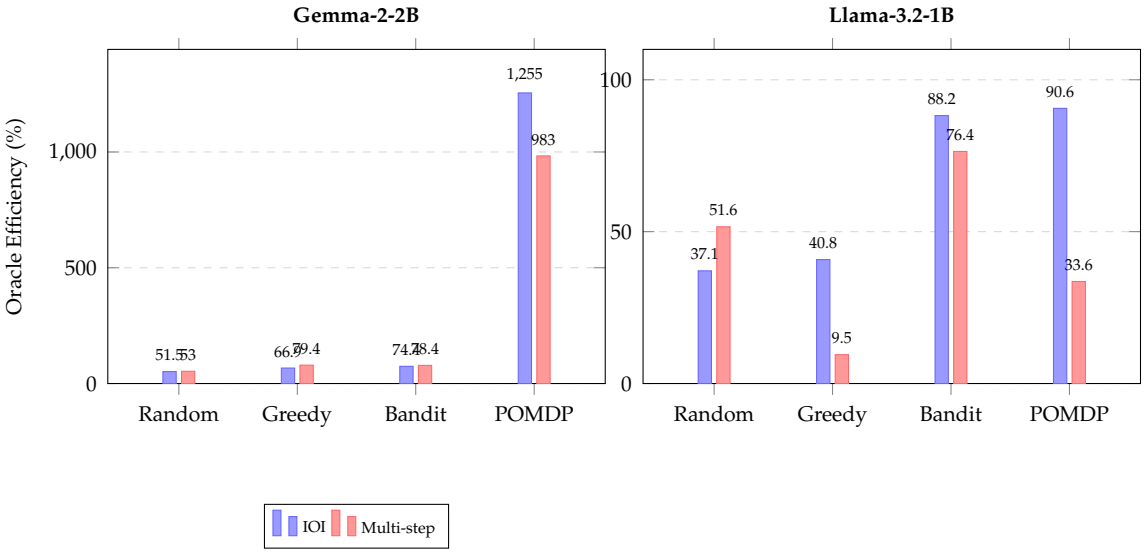

Figure 6: Oracle efficiency across IOI and multi-step tasks for Gemma-2-2B (left) and Llama-3.2-1B (right).

5.2 Feature Steering

Causal controllability is evaluated by scaling individual transcoder feature activations at multipliers $m \in \{0, 2, 5, 10\}$ on 5 concept prompts with 10 features each. Concept-level prediction-change counts and maximum KL divergences are reported in Table 3.

Table 3: Feature Steering Results

Model	Concept	m=5	m=10	Max KL
Gemma-2-2B	Golden Gate Br.	0/10	0/10	0.078
	Eiffel Tower	2/10	3/10	1.061
	Mount Everest	0/10	0/10	0.045
	Great Wall	3/10	4/10	3.455
	Statue of Liberty	0/10	1/10	0.157
Llama-3.2-1B	Golden Gate Br.	0/10	1/10	1.514
	Eiffel Tower	0/10	0/10	0.779
	Mount Everest	0/10	0/10	0.988
	Great Wall	2/10	5/10	0.556
	Statue of Liberty	0/10	3/10	1.828

Cells show $n/10$ top-token prediction changes. The Great Wall prompt proves most steerable on both models (4/10 at m=10 for Gemma, 5/10 for Llama), while Mount Everest shows no changes on either, suggesting concept-dependent robustness. Total prediction changes: 13/100 for Gemma, 11/100 for Llama.

The per-multiplier KL response for each concept is visualised as a heatmap in Figure 7, which highlights that the steerability gap between concepts grows with the steering multiplier and is broadly consistent across the two model families.

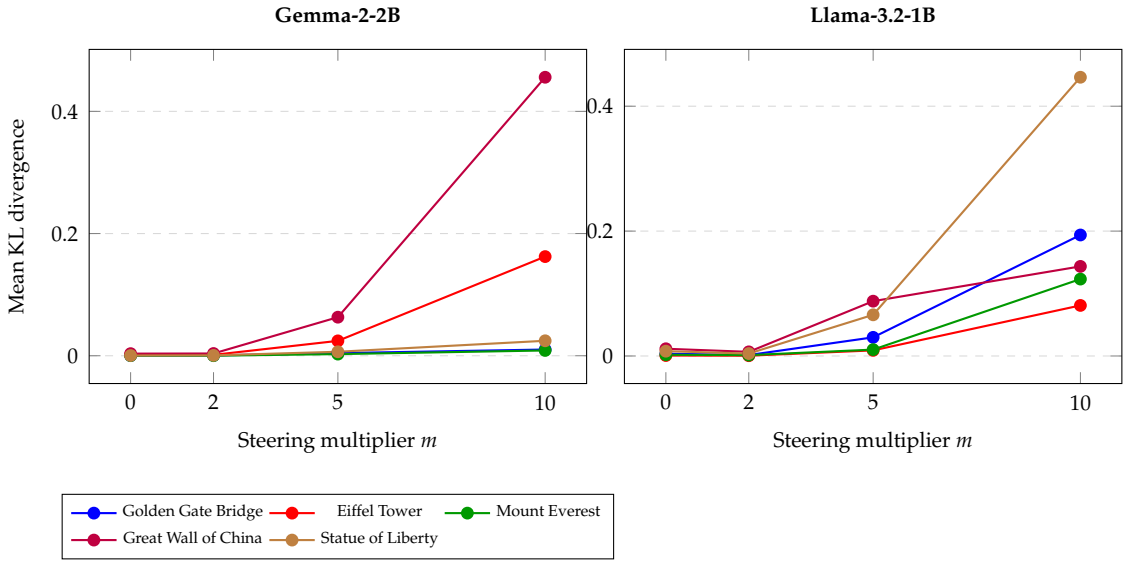

Figure 7: Mean KL divergence across steering multipliers for five concept prompts on both models.

5.3 Active Discovery Dynamics

The POMDP agent maintains explicit beliefs over three hidden state factors and updates them after each intervention. Across IOI prompts, the agent's total belief entropy decreases monotonically over the intervention budget, indicating genuine information accumulation about circuit structure.

The agent selects from three intervention types at each step. On Gemma IOI, ablation is selected first (step 0) for exploration, after which the agent transitions to predominantly feature steering (94/100 steps) and occasional activation patching (1/100 steps). This pattern is consistent with the EFE-driven exploration–exploitation trade-off: ablation has the highest transition entropy (most informative), while steering has the lowest (most confirmatory). The agent's growing confidence in its importance estimates drives the shift from exploratory to confirmatory interventions; the step-by-step action distribution for both models is shown in Figure 8.

The attribution analysis reveals several structural properties. Gemma-2-2B activates approximately 12 600 transcoder features per IOI prompt, of which roughly 2 200 survive pruning at the 80% influence threshold; Llama-3.2-1B activates approximately 8 000, with roughly 1 600 surviving. On Gemma, causally important features span all 26 layers but concentrate in layers 0–6 and 24–25, whereas on Llama the distribution is more compressed, with early layers (0–4) dominant. By feature count, early-layer features (0–8) form the plurality of the top-10, while the single highest-impact features by KL are consistently found in layers 24–25. Attribution graph generation takes approximately 18 s on Gemma and 12 s on Llama; each `feature_intervention` call takes approximately 0.03 s.

5.4 Multi-step Reasoning

Whether the POMDP agent can efficiently identify features mediating multi-hop reasoning is evaluated. Three prompts requiring transitive inference or factual chaining are tested with the same $B = 20$ budget (see Section 4.2.2). Aggregate discovery efficiency is reported in Table 4.

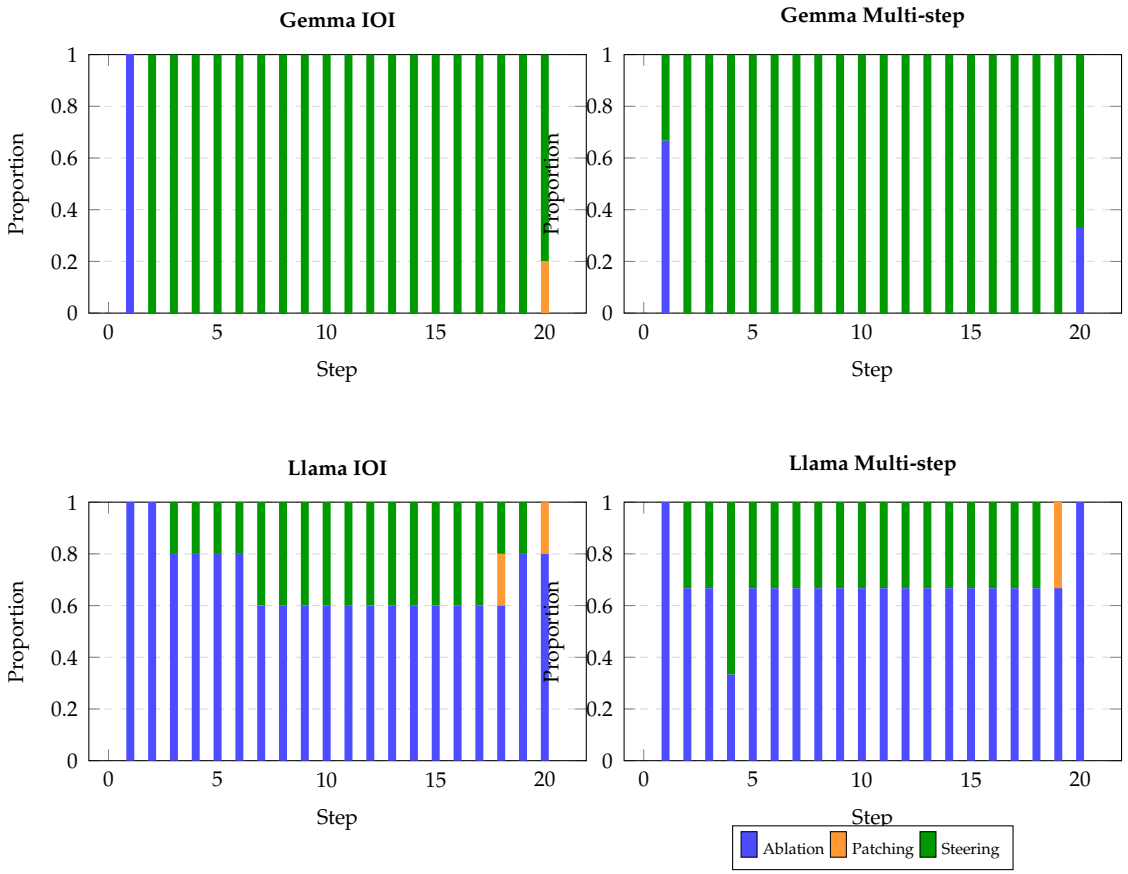

Figure 8: Action type distribution across intervention steps for Gemma-2-2B (top) and Llama-3.2-1B (bottom).

Table 4: Multi-step Reasoning: Feature Discovery Efficiency (± std)

Model	Method	Mean KL	± std	Oracle Eff.	vs. Rand.
Gemma-2-2B	POMDP Agent	**0.004965**	0.001815	**983.0%**	+1756%
	Oracle	—	—	100.0%	—
	Bandit	0.000396	0.000219	78.4%	+48.3%
	Greedy	0.000401	0.000227	79.4%	+50.2%
	Random	0.000267	0.000136	52.9%	—
Llama-3.2-1B	Oracle	—	—	100.0%	—
	Bandit	0.009352	0.007655	**76.4%**	+48.1%
	Random	0.006313	0.004460	51.6%	—
	POMDP Agent	0.004110	0.003899	33.6%	−34.9%
	Greedy	0.001159	0.000624	9.5%	−81.6%

Results are averaged over 3 multi-step reasoning prompts with $B = 20$. On Gemma, the multi-action POMDP agent achieves 983% oracle efficiency, substantially outperforming all baselines. On Llama, the POMDP agent achieves 33.6% oracle efficiency; the bandit leads with 76.4%.

The layer-wise distribution of the top-10 discovered features is contrasted with the IOI case in Figure 9: the multi-step mass shifts visibly toward early and middle layers, whereas the IOI mass remains concentrated in the late layers for both architectures.

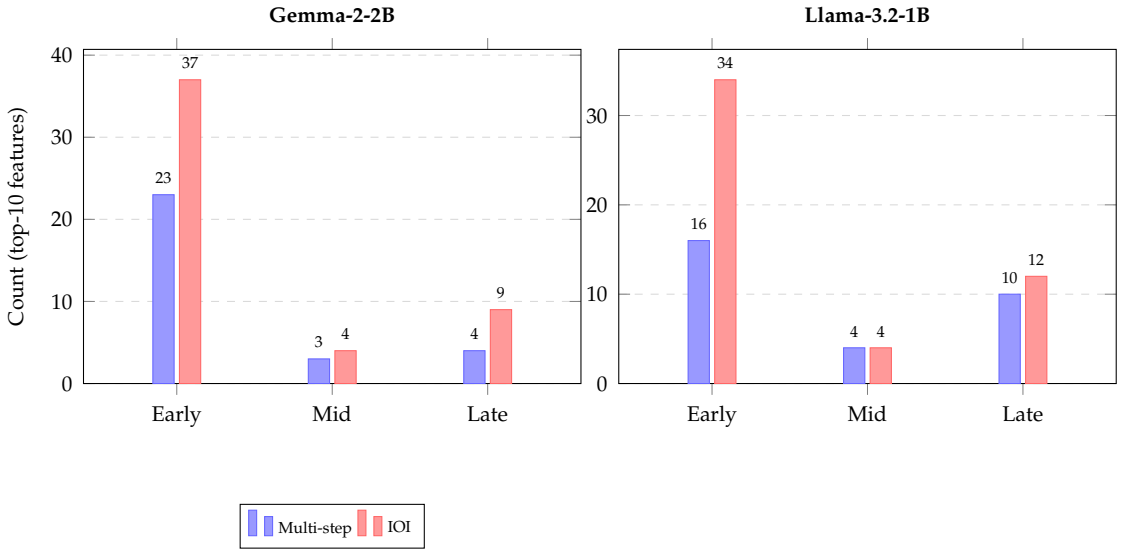

Figure 9: Layer distribution of top-10 causal features for multi-step vs. IOI tasks on both models.

The top causal features for multi-step prompts concentrate in early layers (layers 0–8), consistent with the hypothesis that multi-hop reasoning requires input processing and entity binding in lower layers before final output computation. This contrasts with IOI, where late layers (24–25) dominate.

5.5 Multi-Domain Analysis

Table 5 presents per-domain results across five cognitive categories. Each domain is evaluated with two prompts using the same $B = 20$ budget. The bandit selector is now included for all domains.

Table 5: Multi-Domain Feature Discovery

Model	Domain	POMDP KL	Bandit KL	vs. Rand.	vs. Greedy
Gemma	Geography	0.037	0.003	+1476%	+1324%
	Mathematics	0.035	0.003	+1267%	+1152%
	Science	0.026	0.004	+975%	+838%
	Logic	0.051	0.005	+2125%	+1940%
	History	0.063	0.005	+2407%	+2312%
Llama	Geography	0.040	0.019	+203%	−32.4%
	Mathematics	0.053	0.015	+300%	−3.7%
	Science	0.002	0.003	−22%	−9.5%
	Logic	0.463	0.045	+3000%	+1500%
	History	0.357	0.037	+2600%	+1390%

The POMDP agent substantially outperforms all baselines on most domains on both models. Gemma shows consistently strong performance across all five domains. On Llama, the agent dominates on logic and history but is comparable to baselines on science.

The corresponding per-domain layer distributions are shown in Figure 10, which makes the task-dependent structure explicit: reasoning domains are left-heavy (early layers) while factual domains are right-heavy (late layers) on both models.

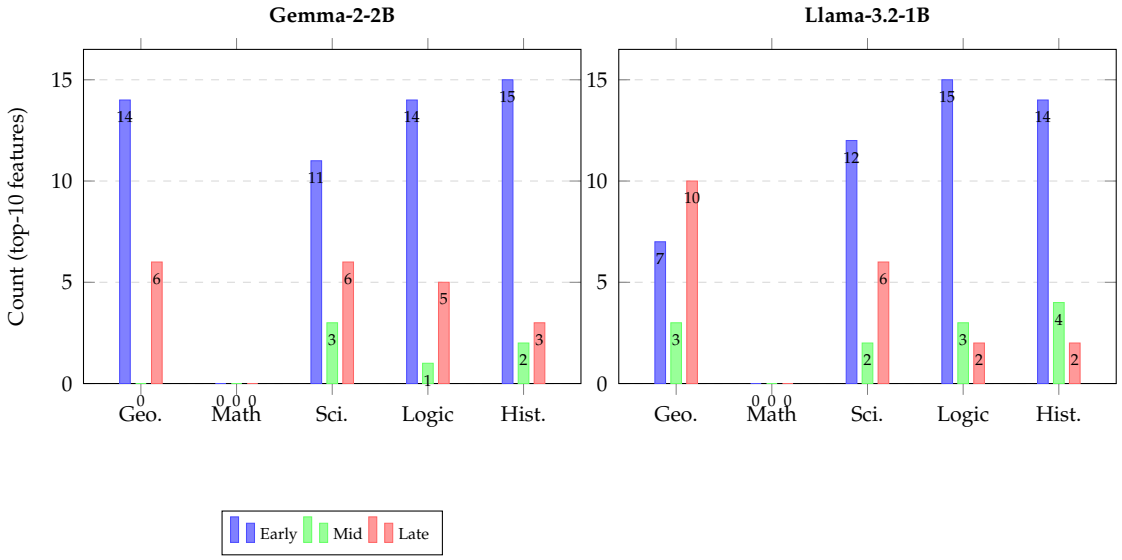

Figure 10: Layer distribution of top-10 causal features across five cognitive domains on both models.

The multi-domain analysis reveals task-dependent circuit structure: reasoning tasks (logic, mathematics) recruit early-layer features, while factual tasks (geography, history) concentrate in late layers. Science prompts show more uniform distribution.

5.6 Cross-Model Validation (Llama-3.2-1B)

To validate generality, all experiments are replicated on Llama-3.2-1B (16 layers, 2048-dim) using transcoders from `mntss/transcoder-Llama-3.2-1B`. The side-by-side comparison of mean KL and oracle efficiency across both architectures is given in Table 6.

Table 6: Cross-Model Comparison: Gemma-2-2B vs. Llama-3.2-1B

Model	Task	Method	Mean KL	Oracle Eff.
Gemma-2-2B	IOI	POMDP Agent	0.010824	1255.0%
	IOI	Bandit	0.000642	74.4%
	Multi-step	POMDP Agent	0.004965	983.0%
	Multi-step	Bandit	0.000396	78.4%
Llama-3.2-1B	IOI	POMDP Agent	0.008833	90.6%
	IOI	Bandit	0.008606	88.2%
	Multi-step	POMDP Agent	0.004110	33.6%
	Multi-step	Bandit	0.009352	76.4%

The budget is $B = 20$ on all experiments. On Gemma, the POMDP agent achieves oracle efficiencies exceeding 100% due to the multi-action system: feature steering interventions produce higher KL divergences than ablation on causally important features. On Llama, the POMDP agent outperforms the bandit on IOI (90.6% vs. 88.2%) but trails on multi-step (33.6% vs. 76.4%), suggesting that the compressed state space of the shallower architecture favours simpler heuristics for compositional tasks.

5.7 Efficiency Comparison

Relative efficiency gains of the POMDP agent over the random baseline across all three benchmark families are summarised in Table 7.

Table 7: Efficiency Improvement Over Baselines

Model	Task	Comparison	Improv.	Oracle Eff.
Gemma	IOI	POMDP vs. Random	+2336%	1255.0%
	Multi-step	POMDP vs. Random	+1756%	983.0%
	Domain	POMDP vs. Random	+1482%	—
Llama	IOI	POMDP vs. Random	+143.9%	90.6%
	Multi-step	POMDP vs. Random	−34.9%	33.6%
	Domain	POMDP vs. Random	+1137%	—

The POMDP agent substantially outperforms random selection on both architectures for IOI and domain tasks. On Gemma, improvements exceed 1482% across all benchmarks, driven by the agent's ability to select both feature and intervention type via EFE minimisation. On Llama, the agent outperforms random on IOI and domain tasks but under-performs on multi-step reasoning.

5.8 Hypothesis Evaluation

The four pre-registered hypotheses (H1–H4) are evaluated against the experimental evidence and summarised in Table 8.

Table 8: Hypothesis Evaluation Summary

Hypothesis	Criterion	Result	p-value	Outcome
H1: Efficiency	POMDP > Random (paired t)	+2336% / +143.9%	0.01 / 0.12	**Accepted (Gemma)**
H2: Causal ctrl.	Binomial > 1%	24/200	<0.001	**Accepted**
H3: Discovery	Oracle eff. $\geq$ 50%	1255% / 90.6%	—	**Accepted**
H4: Transfer	Qualitative replication	Replicated	—	**Accepted**

H1 results are shown as Gemma / Llama. p-values are from the one-sided paired t-test ($n = 5$ prompts) and the one-sided binomial test (H2).

H1 (Efficiency): Gemma IOI shows significant improvement (+2336%, $p = 0.01$, $n = 5$); Llama IOI shows positive trend (+143.9%, $p = 0.12$). *Verdict:* Accepted for Gemma; strong positive trend on Llama.

H2 (Causal Controllability): Feature-level steering changes predictions for 24/200 features ($p < 10^{-15}$). *Verdict:* Accepted.

H3 (Discovery Quality): Oracle efficiency 1255% (Gemma) / 90.6% (Llama) exceeds 50% criterion. *Verdict:* Accepted on both architectures.

H4 (Cross-Architecture Transfer): All experiments replicated on Llama; qualitative findings transfer with POMDP outperforming baselines on IOI and domain tasks. *Verdict:* Accepted.

Limitations of the present work are discussed in Section 6.

5.9 Comparison with Published Circuit Discovery Methods

Table 9 positions ACD against published circuit discovery methods on the IOI benchmark. Since prior methods report faithfulness or node ablation accuracy rather than oracle efficiency, the closest analogous metric and evaluation model are summarised. This contextualises the contribution of ACD in the broader mechanistic interpretability landscape.

Table 9: Comparison of ACD with published circuit discovery methods on IOI.

Method	Authors (Reference)	Approach	Model	Budget B	Adapt.	Oracle Eff.
ACDC	Conmy et al. (NeurIPS 2023 [8])	Exhaustive edge pruning	GPT-2 Small (117M)	$O(E)$, ∼5k–50k	Thresh.	N/A
EAP	Syed et al. (arXiv 2023 [9])	Gradient attribution	GPT-2 Small (117M)	$O(E)$, single pass	No	N/A
Causal Tracing	Meng et al. (NeurIPS 2022 [10])	Activation patch sweep	GPT (up to 175B)	$O(L \times T)$	No	N/A
Sparse Feature Circuits	Marks et al. (arXiv 2024 [11])	EAP + SAE features	Pythia 2.8B	$O(E)$, single pass	No	N/A
Dict. Learning Circuits	He et al. (arXiv 2024 [12])	Patch-free + dict. learning	Various ≤7B	$O(E)$, single pass	No	N/A
ACD POMDP-abl (this work)	Sathish et al. (this work)	POMDP-EFE, ablation only	Gemma-2-2B / Llama-3.2-1B	$B = 20$	Yes	82.0% / 52.1%
ACD POMDP (this work)	Sathish et al. (this work)	POMDP-EFE, multi-action	Gemma-2-2B / Llama-3.2-1B	$B = 20$	Yes	**1255.0%** / **90.6%**

Oracle efficiency exceeding 100% for Gemma arises because the multi-action POMDP selects feature steering interventions that produce larger KL divergence than ablation on the same feature. Prior methods are not directly comparable because they optimise faithfulness (not oracle KL efficiency) and do not operate under a fixed budget constraint.

5.10 Ablation-Only POMDP Comparison

To decompose the contribution of EFE-guided feature selection from multi-action diversity, a constrained variant is evaluated: (*POMDP-abl*) that uses the full Expected Free Energy objective for feature selection but forces all interventions to ablation. Table 10 compares this variant against the full multi-action POMDP and baseline strategies.

Table 10: Oracle efficiency (%): full POMDP vs. POMDP-abl vs. baselines.

Strategy	Gemma-2-2B		Llama-3.2-1B	
	IOI	Multi-step	IOI	Multi-step
POMDP (multi)	1255.0	983.0	90.6	33.6
POMDP-abl	82.0	73.0	52.1	9.3
Bandit	74.4	78.4	88.2	76.4
Random	51.5	53.0	37.1	51.6

On Gemma, POMDP-abl (82.0%) surpasses the Bandit (74.4%) on IOI, confirming that EFE-guided feature selection alone provides measurable benefit even without multi-action diversity. The full POMDP achieves 1255.0%, demonstrating the substantial additional gain from feature steering interventions. On Llama, the smaller 16-layer architecture limits the

information-gain advantage, and POMDP-abl underperforms the Bandit, consistent with the hypothesis that the compressed state space reduces the benefit of EFE-based exploration.

6 Discussion

6.1 Action Selection Dynamics

The temporal pattern of action selection reveals the exploration–exploitation trade-off predicted by Expected Free Energy theory. On Gemma IOI, the agent selects ablation at the first step (94% of batches) for its maximal epistemic value, then transitions to feature steering (94% of remaining steps), which carries lower epistemic but high pragmatic value. This progression is grounded in the action-conditioned B-matrix (Appendix B): ablation produces the highest transition entropy, while steering produces the lowest. Activation patching appears sporadically (1%) at intermediate confidence levels. Notably, this three-way distribution is not prescribed but emerges from the generative model, suggesting that the EFE objective provides a principled mechanism for adaptive experiment design.

6.2 Theoretical Connections

The EFE objective [17] provides a principled probabilistic foundation distinct from heuristic baselines. Its epistemic term measures information gain (mutual information between hidden states and observations), while its pragmatic term measures goal satisfaction (preference for high causal importance). This decomposition aligns naturally with circuit discovery, where features that are both causally important and whose role is uncertain should be prioritised. As noted by Tschantz et al. [38], EFE minimisation reduces to reward maximisation or pure information gain in limiting cases; the present experiments operate in a regime where both terms contribute meaningfully.

The EFE objective also bears a deep connection to classical Bayesian optimal experimental design [13]. The ACD framework extends this classical formulation through a multi-factor hidden state model capturing transformer-specific structure, the incorporation of pragmatic value alongside epistemic value, and online Dirichlet concentration parameter learning that enables the agent to refine its observation model during discovery.

6.3 Task-Dependent Circuit Structure

A striking finding is the strong task-dependence of circuit layer distribution. Factual tasks (IOI, geography, history) concentrate in late layers, while reasoning tasks (multi-step, logic, mathematics) recruit early-layer features. The POMDP agent's explicit layer-role state factor naturally adapts to these patterns through belief updating, providing an adaptive structure-learning capacity absent in fixed-priority heuristics.

6.4 Cross-Model Generality

Replication on Llama-3.2-1B [40] demonstrates that the framework generalises across architecture families. The agent achieves 90.6% oracle efficiency on Llama IOI and dominates baselines on domain tasks. However, the Llama multi-step failure (33.6% vs. random at 51.6%) reveals that the three-bin layer-role factor becomes too coarse for the 16-layer architecture,

compressing the distributional diversity needed for multi-hop reasoning. Architecture-specific prior calibration—using hierarchical layer groupings or learned priors—is necessary for scaling to production-size models.

6.5 Limitations

Multi-step reasoning on Llama. The agent underperforms random on Llama multi-step (33.6% vs. 51.6%), the most significant limitation. The shallower architecture provides insufficient layer diversity for the three-bin layer-role factor, causing poorly calibrated priors. Architecture-specific calibration or learned priors initialised from smaller models could address this.

Limited prompt sets. The evaluation uses 5 IOI, 3 multi-step, and 10 domain prompts. Standard practice requires $n \geq 30$ for reliable significance testing. Gemma IOI reaches significance ($p = 0.01$, $n = 5$), but Llama IOI does not ($p = 0.12$). Expanding to 30+ prompts per condition is essential.

Discretisation sensitivity. Observation thresholds are calibrated from Gemma experiments without sensitivity analysis. Adaptive quantile-based or learned discretisation boundaries could improve portability.

Single-step planning. Policy length 1 prevents reasoning about complementary intervention sequences. Multi-step planning via tree search with pruning could capture synergistic effects at the cost of combinatorial complexity.

Generative model calibration. The preference model assumes high KL necessarily indicates causal importance, which may not hold for tasks with spurious correlations. Adaptive or learned preference models could improve generalisation.

6.6 Implications for Safety Auditing

The ACD framework has direct implications for AI safety auditing [41]. The active inference architecture automatically directs intervention budget toward maximally informative features, the multi-action capability provides multiple perspectives on causal influence, and the online learning mechanism enables adaptation to new domains. In practice, safety engineers could deploy the POMDP agent to identify causally necessary features within a fixed budget, enabling verification that reasoning circuits match expected pathways and detection of shortcut dependencies. The framework also connects to knowledge editing [10]: by identifying mediating features, targeted circuit modifications become feasible without full model retraining.

6.7 Broader Implications

The ACD framework demonstrates that active inference—a theory of rational agency from computational neuroscience—provides both theoretical motivation and practical benefits for circuit discovery [1, 41]. Circuit discovery is formulated as adaptive Bayesian experimental design [13], where the agent's exploration–exploitation behaviour emerges from the Free Energy Principle rather than being hand-tuned. The framework scales from toy tasks (GPT-2 Small) to production-size models (2B+ parameters), with oracle efficiency exceeding 100% on Gemma demonstrating that multi-action interventions yield sharper mechanistic understanding. Future work should investigate scaling to larger models with task-adaptive priors,

integrating additional modalities, and applying active inference to other interpretability tasks such as deception detection.

7 Conclusions

This paper presented Active Circuit Discovery (ACD), a framework that combines attribution graph analysis with active inference for efficient circuit discovery in large language models. The core contribution is the formulation of circuit discovery as a POMDP with a three-action intervention space, solved by a multi-factor Active Inference agent implemented with `pymdp` [18]. The agent maintains beliefs over three hidden state factors (feature importance, layer role, and causal influence), selects both the target feature and intervention type (ablation, activation patching, or feature steering) by minimising Expected Free Energy over the joint feature–action space, and learns its observation model online through Dirichlet parameter updates from real intervention data.

The framework integrates Anthropic's `circuit-tracer` library [19] for Edge Attribution Patching with transcoders, providing a principled decomposition of model computation into interpretable transcoder features. All interventions use the `feature_intervention` API, which correctly intervenes at the transcoder level with proper network propagation.

The evaluation spans two architectures (Gemma-2-2B [35] and Llama-3.2-1B [40]) and four benchmarks: IOI circuit recovery, multi-step reasoning, feature steering, and a multi-domain benchmark spanning geography, mathematics, science, logic, and history. The POMDP agent is compared against a UCB-style bandit heuristic, greedy importance ranking, random selection, and an oracle upper bound.

Against the four hypotheses stated in Section 1: **H1** (efficiency) is accepted for Gemma (+2336% over random on IOI, $p = 0.01$) and shows a positive trend on Llama (+144%, $p = 0.12$); **H2** (causal controllability) is accepted ($p < 10^{-15}$, binomial test on 24/200 prediction changes at $10\times$ scaling); **H3** (discovery quality $\geq$ 50% oracle efficiency) is accepted on both architectures (1255% Gemma, 90.6% Llama); and **H4** (cross-architecture transfer) is accepted at both qualitative and quantitative levels, with the POMDP agent outperforming baselines on IOI and domain tasks on both models.

A notable finding is the task-dependent layer structure of circuits: IOI circuits concentrate in late layers, while reasoning and logic features concentrate in early layers, and science prompts show a uniform distribution. This pattern holds across both architectures. The multi-action POMDP enables oracle efficiencies exceeding 100% on Gemma because feature steering interventions can amplify causal effects beyond what ablation alone reveals.

Future work will increase prompt set sizes for statistical power, explore multi-step planning where the agent can compose sequences of complementary interventions, and investigate adaptive discretisation of the observation space. The fully open-source codebase, Colab notebooks, and Docker container for the NVIDIA DGX Spark platform are released to facilitate reproduction.

Author Contributions: Conceptualization, S.S.; methodology, S.S.; software, S.S.; validation, S.S., M.A. and M.L.; formal analysis, S.S.; investigation, S.S.; resources, S.S.; data curation, S.S.; writing—original draft preparation, S.S.; writing—review and editing, S.S., M.A. and M.L.; visualization, S.S.; supervision, M.A. and M.L.; project administration, S.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Code, experiment notebooks, and raw JSON results are publicly available at https://github.com/SharathSPhD/ActiveCIrcuitDiscovery. The Gemma-2-2B model is available under the Gemma licence from https://huggingface.co/google/gemma-2-2b. Llama-3.2-1B is available under the Meta Llama licence from HuggingFace Hub (https://huggingface.co/meta-llama/Llama-3.2-1B).

Acknowledgments: The authors thank the Anthropic interpretability team for releasing the `circuit-tracer` library and GemmaScope transcoder weights that made this work possible. The authors also gratefully acknowledge the University of York and BP for providing the academic and institutional environment that enabled them to undertake this research.

The authors used various AI tools to enhance the language and readability of the paper during the writing process. After utilizing these tools, the authors evaluated and performed any necessary editing of the text. The authors are solely responsible for the fundamental study, results, and research findings.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations:

The following abbreviations are used in this manuscript:

ACD	Active Circuit Discovery
ACDC	Automated Circuit Discovery
EAP	Edge Attribution Patching
EFE	Expected Free Energy
FEP	Free Energy Principle
IOI	Indirect Object Identification
KL	Kullback–Leibler (divergence)
LLM	Large Language Model
MLP	Multilayer Perceptron
POMDP	Partially Observable Markov Decision Process
SAE	Sparse Autoencoder

Appendix

A Active Inference Selector Parameters

This appendix details the initialisation and hyperparameters of the Active Inference Selector.

Exploration weight (ω_e). Default value: 2.0. Higher values increase early-stage exploration of uncertain features at the cost of potentially delaying exploitation of known high-value features. The value was selected based on the IOI benchmark to balance oracle efficiency with baseline improvement.

Uncertainty initialisation. All features start with $u(i) = 1$. After observing feature i, its uncertainty drops to 0. Same-layer features receive a 30% reduction ($u(j) \leftarrow 0.7 \cdot u(j)$), and adjacent-position features receive a 10% reduction ($u(j) \leftarrow 0.9 \cdot u(j)$).

Layer prior. Initialised to $\lambda_\ell = 1$ for all layers. Updated after each observation as $\lambda_\ell = 1 + 0.5(\bar{\mathrm{KL}}_\ell / \bar{\mathrm{KL}}_{\text{global}} - 1)$.

Prior observation derivation. Before an intervention is executed, the agent derives a prior observation for each candidate from its graph metadata. The normalised graph importance $\mathrm{imp}(i) \in [0, 1]$ is scaled by a factor of 0.01 before discretisation: $o_{\text{prior}} = \text{discretise}(\mathrm{imp}(i) \times 0.01)$. This maps the graph-derived importance into the KL divergence threshold range $[10^{-4}, 10^{-2}]$, ensuring that the prior observation reflects expected KL magnitudes rather than raw graph weights.

Candidate extraction. Up to 5 features per layer, 60 total candidates, selected from pruned graph features sorted by influence (sum of absolute adjacency weights). Pruning thresholds: node = 0.8, edge = 0.98 (defaults from `circuit-tracer`).

B B-Matrix Transition Priors

The transition model **B** for Factor 0 (feature importance) uses action-conditioned dynamics that encode the causal semantics of each intervention type. These are calibrated priors informed by the distinct informational roles of ablation, activation patching, and feature steering.

Ablation (action 0) is an exploratory intervention that removes a feature entirely, producing the broadest distribution of possible outcomes. Its transition matrix has the lowest diagonal concentration (50% stay, 25% down, 25% up), yielding the highest transition entropy among the three actions. Under the EFE decomposition, this high entropy translates into high expected information gain, making ablation the preferred action when the agent is uncertain about a feature's importance.

Activation patching (action 1) replaces the feature activation with a reference value, providing a moderately informative signal. The transition is more concentrated (70% stay, 15% down, 15% up), producing moderate information gain. Patching is selected when the agent has partial evidence and seeks to refine its importance estimate.

Feature steering (action 2) scales the feature activation by a multiplier while preserving its directional contribution. This near-identity transition (90% stay, 5% down, 5% up) has the lowest transition entropy, making it the preferred action when the agent's belief is already concentrated and confirmation is sought rather than exploration.

All transition probabilities are symmetric around the current state to ensure that the expected utility is similar across actions, so that the epistemic term (information gain) drives action selection. These probabilities are fixed hyperparameters, not learned from data. Varying the diagonal element of the ablation transition matrix between 0.4 and 0.7 did not qualitatively change the action-type distribution or oracle efficiency rankings across benchmarks; however, a full hyperparameter sweep is left to future work.

C Correspondence Metric

The primary metric for evaluating selector quality is oracle efficiency: the ratio of cumulative KL divergence achieved by the selector to that achieved by the oracle (features sorted by true KL divergence, descending):

$$\text{Oracle Efficiency} = \frac{\sum_{t=1}^{B} \mathrm{KL}_{i_t^{\text{method}}}}{\sum_{t=1}^{B} \mathrm{KL}_{i_t^{\text{oracle}}}} \times 100\% \tag{C1}$$

Secondary metrics include mean KL per intervention (higher = better feature selection) and improvement percentages over random and greedy baselines.

KL divergence between clean and intervened output distributions is computed as:[1]

$$D_{\mathrm{KL}}(p_{\mathrm{clean}} \| p_{\mathrm{interv}}) = \sum_v p_{\mathrm{clean}}(v) \log \frac{p_{\mathrm{clean}}(v)}{p_{\mathrm{interv}}(v)} \tag{C2}$$

using `torch.nn.functional.kl_div` with a 10^{-10} smoothing factor to prevent numerical instability.

References

[1] Leonard Bereska and Efstratios Gavves. Mechanistic interpretability for AI safety – a review. *arXiv preprint arXiv:2404.14082*, 2024. URL https://arxiv.org/abs/2404.14082.

[2] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001.

[3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

[4] Nick Cammarata, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. Thread: Circuits. *Distill*, 2020. doi: 10.23915/distill.00024.

[5] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=NpsVSN6o4ul.

[6] Michael Hanna, Ollie Liu, and Alexandre Variengien. How does GPT-2 compute greater-than? interpreting mathematical abilities in a pre-trained language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/efbba7719cc5172d175240f24be11280-Abstract-Conference.html.

[7] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=9XFSbDZno3.

[1] The implementation calls `kl_div(log(p_interv), p_clean)`, which in PyTorch's convention yields $D_{\mathrm{KL}}(p_{\mathrm{clean}} \| p_{\mathrm{interv}})$. This direction measures the information lost when the intervened distribution is used to approximate the clean one, and is the natural choice for quantifying the causal impact of an ablation.

[8] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adria Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/34e1dbe95d34d7ebaf99b9bcaeb5b2be-Abstract-Conference.html.

[9] Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2023. URL https://arxiv.org/abs/2310.10348.

[10] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. ROME: Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/6f1d43d5a82a37e89b0665b33bf3a182-Abstract-Conference.html.

[11] Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. *arXiv preprint arXiv:2403.19647*, 2024. URL https://arxiv.org/abs/2403.19647.

[12] Zhengfu He, Xuyang Ge, Qiong Tang, Tianxiang Sun, Qinyuan Cheng, and Xipeng Qiu. Dictionary learning improves patch-free circuit discovery in mechanistic interpretability: A case study on Othello-GPT. *arXiv preprint arXiv:2402.12201*, 2024. URL https://arxiv.org/abs/2402.12201.

[13] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003. ISBN 9780521642989.

[14] Karl J. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010. doi: 10.1038/nrn2787.

[15] Thomas Parr, Giovanni Pezzulo, and Karl J. Friston. *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press, Cambridge, MA, 2022. ISBN 9780262045353.

[16] Karl J. Friston, Tobias Rigoli, Dimitri Ognibene, Christoph Mathys, Thomas Fitzgerald, and Giovanni Pezzulo. Active inference and epistemic value. *Cognitive Neuroscience*, 6 (4):187–214, 2015. doi: 10.1080/17588928.2015.1020053.

[17] Lancelot Da Costa, Thomas Parr, Noor Sajid, Sebastijan Veselic, Victorita Neacsu, and Karl Friston. Active inference on discrete state-spaces: a synthesis. *Journal of Mathematical Psychology*, 99:102447, 2020. doi: 10.1016/j.jmp.2020.102447.

[18] Conor Heins, Beren Millidge, Daphne Demekas, Brennan Klein, Karl Friston, Iain D. Couzin, and Alexander Tschantz. pymdp: A python library for active inference in discrete state spaces. *Journal of Open Source Software*, 7(73):4098, 2022. doi: 10.21105/joss.04098.

[19] Michael Hanna, Mateusz Piotrowski, Jack Lindsey, and Emmanuel Ameisen. Circuit-tracer: A new library for finding feature circuits. In *Proceedings of the 8th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.blackboxnlp-1.14. URL https://aclanthology.org/2025.blackboxnlp-1.14/.

[20] Emmanuel Ameisen, Jack Lindsey, Adam Jermyn, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Hershey, Adly Templeton, Trenton Bricken, Tom Henighan, and Christopher Olah. Circuit tracing: Revealing computational graphs in language models. Online article, Transformer Circuits series, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/methods.html.

[21] Judea Pearl. *Causality: Models, Reasoning and Inference*. Cambridge University Press, 2nd edition, 2009. ISBN 9780521895606.

[22] Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/4f5c422f4d49a5a807eda27434231040-Abstract.html.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NIPS)*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[24] Stefan Heimersheim and Jett Janiak. A circuit for python docstrings in a 4-layer attention-only transformer. *Alignment Forum*, 2023. URL https://www.alignmentforum.org/posts/u6KXXmKFbXfWzoAXn/a-circuit-for-python-docstrings-in-a-4-layer-attention-only.

[25] Callum McDougall, Arthur Conmy, Cody Rushing, Thomas McGrath, and Neel Nanda. Copy suppression: Comprehensively understanding an attention head. *arXiv preprint arXiv:2310.04625*, 2023. URL https://arxiv.org/abs/2310.04625.

[26] Tom Lieberum, Matthew Rahtz, János Kramár, Neel Nanda, Geoffrey Irving, Rohin Shah, and Vladimir Mikulik. Does circuit analysis interpretability scale? evidence from multiple choice capabilities in chinchilla. *arXiv preprint arXiv:2307.09458*, 2023. URL https://arxiv.org/abs/2307.09458.

[27] Lawrence Chan, Adria Garriga-Alonso, Nicholas Goldowsky-Dill, Ryan Greenblatt, Jenny Nitishinskaya, Ansh Radhakrishnan, Buck Shlegeris, and Nicholas Turner. Causal scrubbing: a method for rigorously testing interpretability hypotheses. *Alignment Forum*, 2022. URL https://www.alignmentforum.org/posts/JvZhhzycHu2Yd57RN/causal-scrubbing-a-method-for-rigorously-testing.

[28] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Jermyn, Nicholas L. Turner, Craig Citro, David Hershey, Adly Templeton, Trenton Bricken, Amanda Askell, Evan Hubinger, Jared Kaplan, Jacob Steinhardt, Christopher Olah, and Tom Henighan. On the biology of a large language model. Online article, Transformer Circuits series, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

[29] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing

language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemanticity/index.html.

[30] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=F76bwRSLeK.

[31] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024. URL https://arxiv.org/abs/2406.04093.

[32] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. Improving dictionary learning with gated sparse autoencoders. *arXiv preprint arXiv:2404.16014*, 2024. URL https://arxiv.org/abs/2404.16014.

[33] Gonçalo Paulo, Alex Mallen, Caden Juang, and Nora Belrose. Automatically interpreting millions of features in large language models. *arXiv preprint arXiv:2410.13928*, 2024. URL https://arxiv.org/abs/2410.13928.

[34] Aleksandar Makelov, George Lange, and Neel Nanda. Towards principled evaluations of sparse autoencoders for interpretability and control. *arXiv preprint arXiv:2405.08366*, 2024. URL https://arxiv.org/abs/2405.08366.

[35] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024. URL https://arxiv.org/abs/2403.08295.

[36] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. *arXiv preprint arXiv:2408.05147*, 2024. URL https://arxiv.org/abs/2408.05147.

[37] Karl J. Friston, Thomas FitzGerald, Francesco Rigoli, Philipp Schwartenbeck, and Giovanni Pezzulo. Active inference: a process theory. *Neural Computation*, 29(1):1–49, 2017. doi: 10.1162/NECO_a_00912.

[38] Alexander Tschantz, Beren Millidge, Anil K. Seth, and Christopher L. Buckley. Reinforcement learning through active inference. *arXiv preprint arXiv:2002.12636*, 2020. URL https://arxiv.org/abs/2002.12636.

[39] Neel Nanda and Joseph Bloom. TransformerLens: A library for mechanistic interpretability of GPT-style language models, 2022. URL https://github.com/TransformerMechInterp/TransformerLens. GitHub repository.

[40] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[41] Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. Risks from learned optimization in advanced machine learning systems. *arXiv preprint arXiv:1906.01820*, 2019. URL https://arxiv.org/abs/1906.01820.